

# High-Availability Reference Architecture

**Early Access** · MessageFoundry v0.3.2 · September 2026

MessageFoundry runs as a single process. High availability uses additional engine processes with a shared store. This reference identifies failure domains, layer owners, and components that the operator must supply.

**Read this alongside the Clustering document.** This is the *shape* of an HA deployment. The **mechanism** — leader election, the self-fencing lease, the timing settings, the cluster status endpoints — belongs to the companion **Clustering** document and is not repeated here. Where the two appear to disagree, Clustering is authoritative.

The shared SQL store coordinates the engine tier. One database row identifies the leader. The engine needs no separate broker, quorum service, or arbiter.

SQL Server Always On can require Windows Server Failover Clustering and Active Directory at the database tier. The database administrator manages that cluster.

## Recovery tiers

Three recovery tiers are available. Each covers a different failure domain. None increases processing capacity.

|                                   | Tier 1 — single node                               | Tier 2 — active/passive pair                          | Tier 3 — DR standby                                                        |
|-----------------------------------|----------------------------------------------------|-------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Failure domain covered</b>     | Process failure. The service restarts the process. | loss of an engine host                                | Failure of the engine pair or the whole site.                              |
| <b>Engines</b>                    | one                                                | two or more identical engines, <b>one leader</b>      | one right-sized engine at the recovery site                                |
| <b>Store</b>                      | SQLite, PostgreSQL or SQL Server                   | PostgreSQL <b>or</b> SQL Server, shared by every node | seeded at the recovery site from a backup                                  |
| <b>Capacity after the failure</b> | none until restarted                               | <b>full</b> — the standby runs the same graph         | Reduced capacity. Only feeds at or above the selected priority tier start. |
| <b>Activation</b>                 | operator restart                                   | Automatic. A standby acquires leadership.             | <b>manual and RBAC-gated</b>                                               |
| <b>Operator load balancer</b>     | optional                                           | <b>required</b>                                       | required                                                                   |
| <b>Status</b>                     | the default                                        | opt-in, built                                         | opt-in, built                                                              |

Tier 2 is the supported high-availability topology. It uses active/passive operation. The horizontal active/active model was removed and is not offered.

The configuration loader rejects a node that combines the DR run profile with cluster membership. Use a non-promotable cluster member for a warm recovery-site engine.

### What HA is not: engine shards

An **engine shard** is one of N `serve --shard` processes. Each process owns separate inbound connections and uses one shared store. Shards provide processor parallelism, not redundancy. If a shard fails, its feeds stop.

The service rejects `--shard` when clustering is enabled. The leadership lease covers the whole store and could transfer across shard identifiers. Multiple engine shards require a server database. This topology is **not yet certified for production**. Database sharding is a separate design and is not supported.

## Requirements (Tier 2)

---

A clustered deployment requires, and enforces at configuration load:

- A **server-class shared store** — PostgreSQL or SQL Server. SQLite is a single-file, single-node store and is refused: the cluster needs the shared membership and lease rows a server database provides. Both backends run the same leadership lease and are at parity.
- **Every node points at the same database** (same server, database and schema) and runs the **same** configuration directory.
- A **connection pool with headroom** — a clustered node drives background work (membership and lease renewal, the leader's reclaim sweep, the per-stage workers) alongside request traffic. A pool of one is refused; three or more is the recommendation.

The configuration loader checks all three requirements. Use the same `serve` command on each host. Nodes identify themselves automatically. Pin a node identity only if it must persist across restarts.

## The engine tier — one arbiter, no quorum

---

One row in the shared database holds the **application-level leadership lease**. Only the lease holder runs the message graph. Other nodes remain warm standbys with no listener ports or workers. Each standby maintains its membership heartbeat. When a standby acquires leadership, it starts the full graph.

The database lease has two consequences:

- **There is no quorum and no witness at the engine tier.** The database row is the arbiter, so a two-node cluster is a complete cluster — no odd-vote arithmetic, no third tie-breaker host. Quorum re-enters only if your *database* tier is itself a Windows failover cluster; that is the DBA's design, covered below.
- **There is no node-to-node socket.** Coordination rides the store connection each node already holds, so there is no cluster network to firewall, tune, or encrypt separately.

The Clustering document explains database-clock lease expiry, leader self-fencing, and durable epoch checks during claims. Exactly one node processes messages at a time. Per-lane first-in, first-out ordering continues across failover.

## Keeping leadership at the preferred site

---

By default, the first promotable node to claim an expired lease becomes the leader. A recovery-site leader sends each database commit across the wide-area network. Leadership does not return automatically to the preferred site. Two per-node settings control this behavior:

- **A take-over handicap.** A node can be told to wait a configured number of seconds *past* the lease-expiry instant before it may claim an expired lease. Preferred-site nodes keep the handicap at zero and win the routine race; a remote node takes over only if no preferred node claims. It never delays the sitting leader's renewal and only ever makes a node claim *later*, so it cannot open a two-leader window.
- **A non-promotable standby.** A node can be marked as never eligible for leadership: it never inserts, takes over, or renews the lease, and steps down cleanly if it somehow holds it. That is the right shape for a warm engine at the recovery site. At least one promotable node must exist, or nothing ever leads and the graph never drains.

Choose the preferred site for leadership. Record the fallback procedure in the runbook.

## The client-facing address (operator-supplied)

---

Supply a floating virtual IP address or a Layer-4 load balancer for clients. MessageFoundry does not supply this component or move IP addresses.

Listener health checks identify the active leader:

- **One VIP per inbound port, health-checked with a plain TCP connect** to that listener port.
- **Only the leader binds the port**, so the check passes only on the leader and the VIP lands there automatically.
- **On failover the new leader binds and the old socket closes**, so the VIP follows with no manual intervention.
- **Configure partners to reconnect on drop** — standard MLLP client behaviour. On failover they see a connection drop and reconnect through the VIP.

The engine API — the console and IDE control/read plane — is up on **every** node, so an API-facing VIP can health-check the unauthenticated liveness endpoint. That endpoint answers on every node and therefore cannot select the leader; to pin an operation to the leader, read the cluster's read-only status endpoints instead. Those require a monitoring permission, so a load balancer probing them needs a least-privilege token injected — an unauthenticated probe is rejected everywhere and black-holes traffic. The web console surfaces the current role, leader and lease holder from the same endpoints.

**Not built.** An *engine-managed* VIP, where the engine binds and releases the address itself in lockstep with the lease, is a recorded design (Windows-only at v1) with **no code today**. Architect for the external VIP or load balancer: it is the cross-platform path, and it remains the recommendation for the strictest split-brain guarantee even after an engine-managed option ships.

## The database tier is delegated to the database

MessageFoundry does **not** replicate the store, and on a server database it does not back it up either. **Database-tier availability is the database's job**, using its own mechanism:

- **PostgreSQL** — streaming replication with your chosen failover tooling.
- **SQL Server** — Always On availability groups.

Point every engine node at the database's HA endpoint (the listener or failover address). Engine failover and database failover are then **separate axes**: the engine rides through a database failover as a bounded store outage rather than a leadership change, provided that outage stays comfortably shorter than the margin between the heartbeat interval and the self-fence timeout. Nothing acknowledged is lost either way — a failed commit means the sender never got an acknowledgement and still holds the message.

Four points belong in the design rather than the runbook:

- **Synchronous commit belongs inside one site.** The staged pipeline makes several durable commits per message, and within an ordered lane they run serially — so every millisecond added to the commit round trip is paid several times over, per message. A synchronous replica across a WAN link multiplies the link's round trip into each of those commits. Keep synchronous replicas local, keep the remote replica asynchronous, and accept a small, *measured* recovery-point objective there instead.
- **If the database tier is a Windows failover cluster, quorum comes back — theirs, not ours.** An availability-group design wants an odd vote count with a witness outside both data centres, and zero votes at the recovery site so a flaky link can never cost the primary site its quorum. The engine has no dependency on any of it.
- **Cross-subnet listeners need DNS-side care.** The store connection takes one Always On-aware ODBC keyword, and it has shipped: `[store].multi_subnet_failover` emits `MultiSubnetFailover=Yes`, so the driver races the subnets instead of serially waiting each one out. It is **opt-in and defaults to off**, it has not yet been exercised against a live cross-subnet availability group, and it covers the *store* connection only — the separate database-lookup connector builds its own connection string and does not emit it. So set the keyword *and* configure the listener to register only the active subnet's address with a short record TTL, then drill the cross-site connect path.
- **Backups and restores are yours.** On a server database the engine's DR backup covers **configuration only**; restoring the database itself is delegated to the DBA. Promotion at the recovery site is fail-closed on that: it requires an explicit per-activation attestation that the database was restored, plus a live provenance check on the restored store, and refuses otherwise.

## Third tier — the DR standby

---

An optional **DR standby** runs at the recovery site with a restricted profile. It starts fewer feeds than the primary deployment:

- it starts **only** the connections at or above a configured priority tier; everything below reports as filtered rather than starting;
- it **seeds its store from an encrypted backup**, and **fails closed** if the store's key material is not available at the recovery site;
- **activation is manual** — an RBAC-gated, audited operator action; automatic activation is rejected at configuration load;
- it can be fenced by an "acquire the address or abort" hook, so it never starts binding feeds behind a load balancer that has not actually moved.

Use the DR standby when the recovery plan excludes a full-capacity second site. Do not configure it as a cluster member. The configuration loader rejects that combination.

## What a failover actually costs

---

Allow time for standby promotion. Failover is not instantaneous.

- **Clean stop** (planned switchover or graceful shutdown) — the leaving leader releases its lease, so a standby is promoted on its next heartbeat.
- **Crash or network partition** — the dead leader's lease has to age out, and a standby acquires once it expires. A partitioned old leader stops processing before that point, so it never overlaps its successor.

The engine acknowledges messages only after a durable commit. The new leader recovers interrupted work from the previous leader. It recovers each lane's oldest interrupted message before newer messages, preserving order.

**Delivery is at-least-once. Downstream endpoints must be idempotent.** A message interrupted during delivery can be delivered again.

Use the published failover benchmark to estimate the promotion window. Refer to Clustering for lease timing and network-specific settings.

## Operational guidance

---

- **Keep node clocks synced (NTP).** Leadership is decided on the database's clock, so skew cannot change who may lead — but the per-row recovery leases use node wall-clock, so keep the hosts sane.
- **Deploy identical configuration to every node.** Each node loads its graph from its own configuration directory; the cluster coordinates the reload *version*, not the files. A divergent node is not cosmetic — on promotion it can dead-letter messages for destinations it does not know about.
- **Apply configuration changes as a coordinated restart, not a rolling one,** so no two nodes ever run divergent graphs across the change window.

- **Drill the failover you depend on** — a planned database failover, an engine promotion, and, if you run one, the recovery-site activation including the address move and any sender cutover.

## Transport security still applies

HA does not change the transport posture. MessageFoundry binds loopback by default and **refuses at startup** a non-loopback API bind without TLS or a declared trusted terminator; a non-loopback MLLP, HTTP or DICOM listener without TLS is refused the same way. When you expose a listener off loopback — which any real data-plane feed requires — enable TLS on that channel: the API supports in-process TLS (a TLS 1.2 floor, with optional client-certificate mTLS) or an upstream terminator, and MLLP supports per-connection TLS and optional mutual TLS. Raw TCP and X12 listeners have **no TLS of their own**, so a non-loopback bind of one is refused outright — keep them bound to loopback behind a TLS-terminating proxy, or segment them at the OS/firewall level. Inter-node coordination rides the shared store connection and adds **no node-to-node socket**, so securing the store connection secures the cluster traffic with it. This is deployment guidance for the environment, not a limitation of the HA model.

## Related

---

- **Clustering** (the companion document in this set) — the HA mechanism: leader lease, graph gating, lease-timing tuning, and the cluster status endpoints. [CLUSTERING.md](#)
- [AOAG-DEPLOYMENT.md](#) — a worked two-site reference deployment on SQL Server Always On, including placement, quorum and the failure/runbook matrix.
- [DEPLOYMENT.md](#) — network exposure and the per-channel TLS posture matrix.
- [SYSTEM-REQUIREMENTS.md](#) — sizing, measured throughput tiers, and which topologies are supported.
- [CONFIGURATION.md](#) — the full store, cluster and DR settings catalog.